



Video Encoder

OpenMAX Integration Layer API User Manual

Document Revision 0.81

31 December 2023

This document is compatible with the following products:

Hantro H1 Video Encoder V7 or later

Hantro H2 Video Encoder V3 or later

Hantro VC8000E Series Video Encoder

Hantro VC9000E Series Video Encoder

VERISILICON

LEVEL A: PROPRIETARY INFORMATION

Legal Notices

COPYRIGHT INFORMATION

This document contains proprietary information of VeriSilicon Holdings Co., Ltd. VeriSilicon reserves the right to make changes to any products herein at any time without notice. VeriSilicon does not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by VeriSilicon; nor does the purchase or use of a product from VeriSilicon convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of VeriSilicon or third parties.

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained in this VeriSilicon proprietary copyright document may be re-distributed by VeriSilicon customer SoC vendors who have licensed the related Hantro IP, with the understanding that the material is re-packaged/re-branded as a licensee (SoC vendor) written/labeled document. Those portions of the SoC vendor document adapted from the VeriSilicon Hantro material should contain a notice similar to: "This section contains copyright material disclosed with permission of VeriSilicon Holdings Co., Ltd."

(VeriSilicon Distribution **Level A: PROPRIETARY INFORMATION**)

TRADEMARK ACKNOWLEDGMENT

VeriSilicon, the VeriSilicon logo, Hantro and the Hantro logo are the trademarks of VeriSilicon Holdings Co., Ltd. in the United States and/or other jurisdictions. All other trademarks are the property of their respective holders.

For our current distributors, sales offices, design resource centers, and product information, visit our web site located at <http://www.verisilicon.com>.

VeriSilicon Confidential, Copyright © 2023 by VeriSilicon Holdings Co., Ltd. All rights reserved worldwide.

Preface

This preface introduces the abbreviations used in this document and provides related documents for your reference.

Glossary

Term	Description
API	Application programming interface
AVC	Advanced Video Coding (that is, H.264 or MVC)
AVS	Audio Video Coding Standard of China
CABAC	Context adaptive binary arithmetic coding, HEVC entropy coding method
CAVLC	Context-adaptive variable-length coding
CIR	Cyclic intra refresh
codec	Compressor/decompressor
CPB	Coded picture buffer
CTB	Coding tree block
CTU	Coding tree unit
CU	Coding unit
DC	Direct current
DivX	A brand name of products created by DivX, Inc. The DivX codec uses MPEG-4 compression.
GDR	Gradual decoding refresh
GOP	Group of pictures
H.263/H.264	ITU-T video coding standards
HEVC	High Efficiency Video Coding, a video coding standard (JCT-VC) (that is, H.265)
HRD	Hypothetical reference decoder

Term	Description
HW	Hardware
IDR	Instantaneous decoder refresh
IL	OpenMAX integration layer
IPCM	Intra pulse code modulation
IRQ	Interrupt request
JPEG	Joint Photographic Experts Group, also a common term for still images
LTR	Long-term reference
MB	Macroblock, a data unit of four 8x8 luminance pixel blocks, one 8x8 Cb block, and one 8x8 Cr block
MPEG-4	Motion Picture Experts Group standard 4 (ISO/IEC 14496-2)
MVC	Multiview Video Coding
NAL	Network Abstraction Layer (in H.264)
OMX	OpenMAX multimedia interface developed and maintained by the Khronos group
OS	Operating system
OSAL	Operating system abstraction layer, a part of OpenMAX conformance testing package
Planar	A YCbCr storage format, where Y/Cb/Cr samples separately form their independent plane in memory
POC	Picture order count
PPS	Picture parameter set
PSNR	Peak signal-to-noise ratio
QP	Quantization parameter
RC	Rate control
RDO	Rate-distortion optimization
RDOQ	Rate-distortion optimized quantization

Term	Description
RGB	Red, green, blue color space representation
ROI	Region of interest
RPS	Reference picture set
SAO	Sample adaptive offset
Semi-planar	A YCbCr storage format, where the luminance samples form one plane in memory, and the pixel by pixel interleaved Cb and Cr samples form another
SPS	Sequence parameter set
SSIM	Structural similarity
UUID	Universal unique identifier
VBR	Variable bit rate
VC-1	A Microsoft SMPTE 421M video codec
VP6	TrueMotion VP6 video coding standard, developed by On2 Technologies
VP8	A compressed video data format developed by On2 Technologies
VP9	An open and royalty free video compression standard developed by Google
VUI	Video usability information
WebP	VP8-based still image compression
YCbCr	Color space with one luma and two chroma components used for digital encoding
YCbCr 4:2:0	YCbCr sampling format, where Cb and Cr components are sub-sampled by two both horizontally and vertically
YCbCr 4:2:2	YCbCr sampling format, where Cb and Cr components are sub-sampled by two horizontally
YUV	Color space with one luma and two chroma components, originally used for analog encoding and currently often used for either digital or analog encoding

References

- [1] OpenMAX Integration Layer Application Programming Interface Specification v1.1.2, The Khronos Group, 2008.
- [2] ISO/IEC FDIS 14496-10: Information technology – Coding of audio-visual objects – Part 10: Advanced Video Coding.
- [3] ISO/IEC IS 10918-1: Information technology – Digital compression and coding of continuous-tone still images, Oct 20, 1999.



Table of Contents

Legal Notices.....	2
Preface	3
Glossary	3
References	6
Table of Contents.....	7
List of Figures.....	10
List of Tables.....	10
1 Overview	11
1.1 VeriSilicon's Hantro Encoders	11
1.2 OpenMAX IL API for VeriSilicon's Hantro Encoders.....	11
1.2.1 Block Diagram.....	12
1.2.2 OpenMAX Encoder Components.....	12
1.2.3 OpenMAX Component Ports	13
1.3 API Files	13
2 Feature Support.....	14
2.1 Codec Standards	14
2.2 Input Buffers.....	15
2.2.1 Input Color Formats.....	16
2.2.2 Input Payload Formats.....	16
2.2.3 Input Picture Endianness.....	17
2.3 Output Buffer Formats.....	17
2.3.1 Output Compression Formats.....	17
2.3.2 Output Payload Formats.....	18
2.4 Frame Size Limitations.....	18
3 Programming Flow.....	19
3.1 Creating a Component Instance.....	19
3.2 Encoding a Frame.....	20
4 API Components	21
4.1 Buffer Flags.....	21
4.1.1 VP8 Base Layer Frame Flag.....	21
4.1.2 VP8 First Layer Frame Flag	21
4.1.3 VP8 Second Layer Frame Flag.....	21
4.1.4 VP8 Third Layer Frame Flag.....	21

4.2	Enumerations.....	22
4.2.1	OMX_COLOR_FORMATVSITYPE.....	22
4.2.2	OMX_INDEXVSITYPE	22
4.2.3	OMX_VIDEO_AVCEXTLEVELTYPE	23
4.2.4	OMX_VIDEO_CODINGVSITYPE	23
4.2.5	OMX_VIDEO_HEVCLEVELTYPE.....	24
4.2.6	OMX_VIDEO_HEVCPROFILETYPE	25
4.2.7	OMX_VIDEO_VP8LEVELTYPE	25
4.2.8	OMX_VIDEO_VP8PROFILETYPE	25
4.3	Structures	26
4.3.1	OMX_CONFIG_INTRAREFRESHVOPTYPE	26
4.3.2	OMX_CONFIG_FRAMERATETYPE.....	26
4.3.3	OMX_PARAM_COMPONENTROLETYPE	26
4.3.4	OMX_VIDEO_CONFIG_ADAPTIVEROITYPE	27
4.3.5	OMX_VIDEO_CONFIG_AVCINTRAPERIOD	27
4.3.6	OMX_VIDEO_CONFIG_INTRAAREATYPE	27
4.3.7	OMX_VIDEO_CONFIG_IPCMAREATYPE.....	28
4.3.8	OMX_VIDEO_CONFIG_ROIAREATYPE	29
4.3.9	OMX_VIDEO_CONFIG_ROIDELTAQPTYPE	30
4.3.10	OMX_VIDEO_CONFIG_VP8TEMPORALLAYERTYPE	30
4.3.11	OMX_VIDEO_PARAM_AVCEXTTYPE	31
4.3.12	OMX_VIDEO_PARAM_BITRATETYPE.....	32
4.3.13	OMX_VIDEO_PARAM_HEVCTYPE	32
4.3.14	OMX_VIDEO_PARAM_INTRAREFRESHTYPE	45
4.3.15	OMX_VIDEO_PARAM_PROFILELEVELTYPE.....	45
4.3.16	OMX_VIDEO_PARAM_VP8TYPE	46
4.3.17	OMX_VIDEO_VP8REFERENCEFRAMETYPE	46
4.3.18	OUTPUT_BUFFER_PRIVATE	47
5	API Functions	48
5.1	HantroHwEncOmx_encoder_init().....	48
5.2	GetComponentVersion()	49
5.3	SendCommand()	49
5.3.1	Supported Commands	50
5.4	GetParameter().....	51
5.4.1	Supported Structures.....	52
5.5	SetParameter().....	53

5.5.1 Supported Structures.....	54
5.6 GetConfig()	57
5.6.1 Supported Structures.....	58
5.7 SetConfig()	59
5.7.1 Supported Structures.....	60
5.8 GetExtensionIndex().....	61
5.8.1 Supported Extensions	62
5.9 GetState()	62
5.10 ComponentTunnelRequest().....	62
5.11 UseBuffer()	64
5.12 AllocateBuffer().....	65
5.13 FreeBuffer()	66
5.14 FillThisBuffer().....	66
5.15 EmptyThisBuffer()	67
5.16 SetCallbacks().....	68
5.17 ComponentDeInit()	69
Document Revision History.....	70

List of Figures

Figure 1. Block Diagram of OpenMAX Integrated Encoder System	12
Figure 2. OMX Component Ports	13
Figure 3. Creating a Component Instance with OpenMAX IL API	19
Figure 4. Encoding a Frame with OpenMAX IL API	20

List of Tables

Table 1. OpenMAX Encoder Components.....	12
Table 2. Structures Supported for GetParameter() in Video Domain.....	52
Table 3. Structures Supported for GetParameter() in Image Domain	53
Table 4. Structures Supported for GetParameter() in Video Domain.....	54
Table 5. Structures Supported for SetParameter() in Image Domain.....	56
Table 6. Structures Supported for GetConfig() in Video Domain	58
Table 7. Structures Supported for GetConfig() in Image Domain.....	58
Table 8. Structures Supported for SetConfig() in Video Domain	60
Table 9. Structures Supported for SetConfig() in Image Domain	61
Table 10. Supported Extensions.....	62

1 Overview

This document describes VeriSilicon's extension to the OpenMAX Integration Layer (IL) application programming interface (API) for VeriSilicon's Hantro encoders.

Note: This document assumes that the reader understands the fundamentals of C-language and the H.264, HEVC, AV1, VP9, and JPEG standards.

1.1 VeriSilicon's Hantro Encoders

VeriSilicon's Hantro encoders are multi-format encoders based on silicon-proven designs. They provide up to 8192x8640 encoding for H.264 [2] and HEVC, 4096x2304 encoding for AV1 and VP9 video streams, and 16kx16k encoding for JPEG [3] and WebP still images. The overall performance and capabilities of the encoders are system dependent.

Note: Formats and features that are presented depend on the configuration that Licensee has chosen for the encoder hardware product and only formats that are applicable to chosen configuration are supported.

1.2 OpenMAX IL API for VeriSilicon's Hantro Encoders

The OpenMAX IL API for VeriSilicon's Hantro encoders is developed based on the OpenMAX Integration Layer Application Programming Interface Specification v1.1.2 [1], which defines a standardized multimedia component interface that allows easier and faster integration of multimedia components into a multimedia system. It supports OMX IL based profile and tunneling.

The API specifies the functions and their usage for controlling a multimedia object. The data flow from one component to another can be done in either non-tunneled or tunneled way. The API functions should be non-blocking which implies a threaded implementation of the component. The non-blocking commands must be queued to the component. A set of callback functions is used to inform the IL client of the component events.

1.2.1 Block Diagram

Figure 1 describes the blocks of the encoder system with OpenMAX integrated.

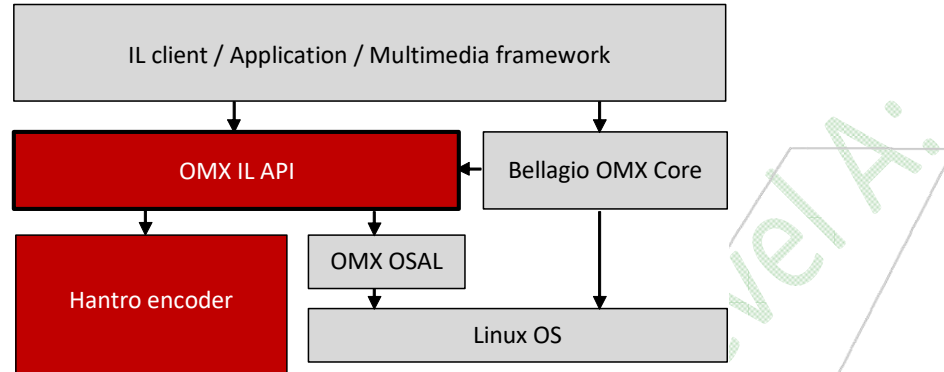


Figure 1. Block Diagram of OpenMAX Integrated Encoder System

As shown in the figure, the OpenMAX IL API for encoders is implemented on top of encoder APIs. The encoder control software runs in its own thread. The OMX API functions queue the data to be handled in the encoder thread. The thread processes the commands, handles the encoding, and issues callbacks and events to the IL client.

The encoder OpenMAX IL API is developed on Linux environment and the implementation uses operating system abstraction layer (OSAL) functions to isolate all OS-specific functionality. This facilitates OS independent implementation of memory allocation, thread, and mutex handling.

The [Bellagio open-source OMX IL Core for Linux](#) is used for development and testing purposes, which handles the component library loading, component creation, and tunnel setup, as defined in the OpenMAX Integration Layer Application Programming Interface Specification v1.1.2 [1]. The OMX Core is responsible for loading the component library and creating the component. The encoder OpenMAX IL API provides an initialization function for the OMX Core to call when creating a component instance. The design also allows flexible integration into other OMX Core systems as well. Integration to other environments requires the OMX Core and OMX OSAL parts to be implemented for the target environment.

1.2.2 OpenMAX Encoder Components

The following table lists the components supported by the OpenMAX IL API for each encoder variant.

Table 1. OpenMAX Encoder Components

Encoder Variant	Video Encoder Component	Image Encoder Component
H1	OMX.hantro.H1.video.encoder	OMX.hantro.H1.image.encoder
H2	OMX.hantro.H2.video.encoder	OMX.hantro.H2.image.encoder
VC8000E	OMX.hantro.VC8000E.video.encoder	OMX.hantro.VC8000E.image.encoder

VC9000E	OMX.hantro.VC9000E.video.encoder	OMX.hantro.VC9000E.image.encoder
---------	----------------------------------	----------------------------------

1.2.3 OpenMAX Component Ports

Each encoder component has an input port and an output port, as shown in [Figure 2](#).

- Port 0 is used for video/image frame input.
For the supported input formats, see [Section 2.2.1, Input Color Formats](#).
- Port 1 is used for video/image stream output.
For the supported output formats, see [Section 2.3.1, Output Compression Formats](#).

For details about parameters and configurations supported for each port, see [Chapter 5, API Functions](#).

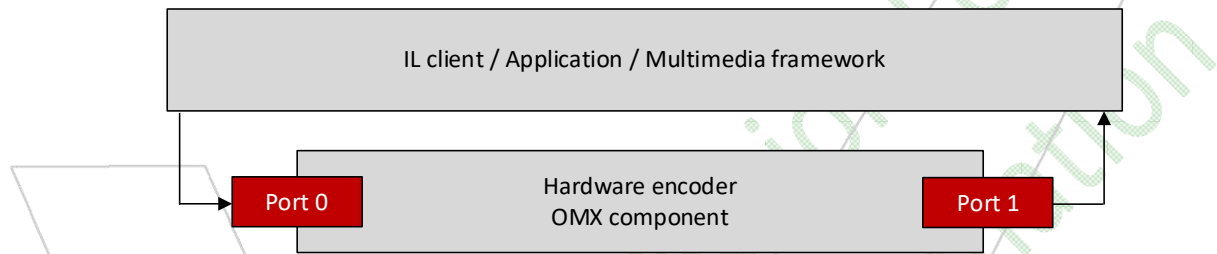


Figure 2. OMX Component Ports

1.3 API Files

The definition files for API components and functions are:

- Component definition files:
 - ./project.omx/openomx_il/source/vsi_vendor_ext.h
 - ./project.omx/openomx_il/source/encoder/encodec.h
- Function definition files:
 - ./project.omx/openomx_il/source/encoder/encoder_constructor.h
 - ./project.omx/openomx_il/headers/*

2 Feature Support

This chapter lists the features supported by VeriSilicon's Hantro encoders.

Note: Feature support and performance vary across encoder variants. For details, see the hardware features document provided in the release package.

2.1 Codec Standards

The following table lists the codec standards supported by each encoder variant.

Encoder Variant	Supported Codec Standards
H1 V7	- AVC (H.264) - ITU-T Rec. H.264 (03/2010), ISO/IEC 14496-10 - Main profile, levels 1 to 5.1 - High profile, levels 1 to 5.1 - Baseline profile, levels 1 to 4.1 - VP8, Main profile, levels 1 to 3 - ITU-T.81 JPEG, Baseline interleaved/JFIF - WebP still images
H2 V3	- HEVC (H.265) - ITU-T Rec. H.265 (04/2013), ISO/IEC 23008-2 9 - Main Profile, level 5.1, High tier
H2 V4.1	- HEVC (H.265) - ITU-T Rec. H.265 (04/2013), ISO/IEC 23008-2 9 - Main profile, level 6.2, High tier - AVC (H.264) - ITU-T Rec. H.264 (03/2010), ISO/IEC 14496-10 - Main profile, levels 1 to 5.1 - High profile, levels 1 to 5.2 - Baseline profile, levels 1 to 4.1
VC8000E	- HEVC (H.265) - ITU-T Rec. H.265 (04/2013), ISO/IEC 23008-2 9 - Main profile, level 1 to 5.1, High tier - AVC (H.264) - ITU-T Rec. H.264 (03/2010), ISO/IEC 14496-10 - Main profile, levels 1 to 5.2 - High profile, levels 1 to 5.2 - Baseline profile, levels 1 to 5.2 - ITU-T.81 JPEG, Baseline interleaved/JFIF

Encoder Variant	Supported Codec Standards
VC9000E	- HEVC (H.265) - ITU-T Rec. H.265 (04/2013), ISO/IEC 23008-2 - Main profile, up to level 6, High tier - Main10 profile, up to level 6, High tier - Main Still Profile, up to level 6 - AVC (H.264) - Spec Version 12: ISO/IEC 14496-10 / ITU-T Rec. H.264 (03/2010) - Baseline Profile, up to level 6 - Main Profile, up to level 6 - High Profile, up to level 6 - High 10 Profile, up to level 6 - JPEG - ISO/IEC 10918-1, ITU-T T.81 - Subset of Baseline Process - DCT-based, 8-bit pixel, sequential, interleaved scans - Huffman coding: 2 AC and 2 DC tables - Up to 3 components - Subset of lossless process (defined in Annex H of T.81) - Predictive process (not DCT-based), 8-bit pixel, sequential, interleaved scans - Huffman coding: 2 DC tables - Up to 3 components - AV1 - AV1 Bitstream & Decoding Process Specification Version 1.0.0 with Errata 1 - Main profile, level 5.1 - VP9 - VP9 Bitstream & Decoding Process Specification v0.6 - Profile 0 (8-bit), level 5.1 - Profile 2 (10-bit), level 5.1

2.2 Input Buffers

It is recommended that the component is allowed to allocate all its input buffers. This can be done by using the OMX API function `AllocateBuffer()`. The component will allocate buffers that are hardware accessible, thus removing any need to copy data from input buffers to internal buffers. Using `UseBuffer()` instead may degrade encoding performance.

2.2.1 Input Color Formats

VeriSilicon's Hantro encoders support the following color formats for input buffers:

- OMX_COLOR_Format16bitARGB4444
- OMX_COLOR_Format16bitARGB1555
- OMX_COLOR_Format16bitRGB565
- OMX_COLOR_Format16bitBGR565
- OMX_COLOR_Format25bitARGB1888
- OMX_COLOR_Format32bitARGB8888
- OMX_COLOR_FormatCbYCrY
- OMX_COLOR_FormatYCbYCr
- OMX_COLOR_FormatYUV420PackedPlanar
- OMX_COLOR_FormatYUV420PackedSemiPlanar

For details about color formats, see the memory buffer and format organization document provided in the release package.

Note: The input color format can be set by calling the `SetParameter()` function to set the `eColorFormat` member of the `OMX_PARAM_PORTDEFINITIONTYPE` structure through port 0.

2.2.2 Input Payload Formats

VeriSilicon's Hantro encoders support following payload formats for input buffers.

1. Full buffer that may consist of several complete frames (the last frame may be partial)
2. Several complete encoding units/frames per buffer
3. One complete encoding unit/frame per buffer

For H.264 NAL unit streams and VC-1, VP8, and VP9 streams, only payload format 2 is supported. This is because these streams do not contain start codes for identifying the start of each encoding unit.

For HEVC streams, only payload format 2 is supported.

For details about the payload formats, see the *Buffer Payload* section in the OpenMAX Integration Layer Application Programming Interface Specification v1.1.2 [1].

The input buffer format and size indicate which payload format is used.

- When the buffer size is not a multiple of frame size, it is assumed to be of payload format 1.
- When input buffer contains partial frames, it is required that the encoder uses internal buffer to store partial frame. Copying partial frame to internal buffer may degrade the encoding performance and thus it is recommended to use the input buffer payload formats 2 and 3 whenever possible.

It is recommended that you choose an input buffer size large enough to fit the biggest encoding unit or the size of one complete uncompressed input frame.

2.2.3 Input Picture Endianness

The input picture data endianness is set when the encoder software is compiled and cannot be changed during run-time. The sizes of the different components are based on the picture dimensions and the input format.

- For planar YCbCr 4:2:0:
 - $Y = \text{width} \times \text{height}$ (bytes)
 - $Cb = \text{width}/2 \times \text{height}/2$ (bytes)
 - $Cr = \text{width}/2 \times \text{height}/2$ (bytes)
- Semi-planar YCbCr 4:2:0:
 - $Y = \text{width} \times \text{height}$ (bytes)
 - $Cb+Cr = \text{width} \times \text{height}/2$ (bytes)
- Interleaved YCbCr 4:2:2:
 - $Y+Cb+Cr = \text{width} \times \text{height} \times 2$ (bytes)
- 16-bit RGB
 - $R+G+B = \text{width} \times \text{height} \times 2$ (bytes)
- 32-bit RGB
 - $R+G+B = \text{width} \times \text{height} \times 4$ (bytes)

2.3 Output Buffer Formats

The dimensions of each output frame are unknown during encoder initialization. In typical cases, the output buffer size can be 1/4 of the input frame size without losing any encoded frames. In the worst case, the encoding does not compress the input frame at all.

2.3.1 Output Compression Formats

The following table lists the codec standards supported by each encoder variant.

Note: The output color format can be set by calling the `SetParameter()` function to set the `eCompressionFormat` member of the `OMX_PARAM_PORTDEFINITIONTYPE` structure through port 1.

Encoder Variant	Supported Output Compression Formats
H1	• OMX_VIDEO_CodingAVC • OMX_VIDEO_CodingVP8 • OMX_IMAGE_CodingJPEG • OMX_IMAGE_CodingWEBP
H2	• OMX_VIDEO_CodingHEVC

Encoder Variant	Supported Output Compression Formats
VC8000E	• OMX_VIDEO_CodingHEVC • OMX_VIDEO_CodingAVC • OMX_VIDEO_CodingAV1 • OMX_IMAGE_CodingJPEG • OMX_IMAGE_CodingWEBP
VC9000E	• OMX_VIDEO_CodingHEVC • OMX_VIDEO_CodingAVC • OMX_VIDEO_CodingAV1 • OMX_IMAGE_CodingJPEG • OMX_IMAGE_CodingWEBP • OMX_IMAGE_CodingVP9

2.3.2 Output Payload Formats

VeriSilicon's Hantro encoders support only one complete frame per buffer as the payload format of output buffers. This means that the encoders store each output frame in separate output buffer. If an encoded frame cannot fit into the output buffer, it is discarded.

2.4 Frame Size Limitations

Limitations regarding the input picture size and the encoded picture size are as follows:

- $\text{encoded_width} \% 4 = 0$
- $\text{encoded_height} \% 2 = 0$
- $\text{input_width} \geq (\text{encoded_width} + 15) \& (\sim 15)$
- $\text{input_width} \% 16 = 0$
- $\text{input_height} \geq \text{encoded_height}$
- $\text{input_height} \% 2 = 0$

where % is the remainder operator, & is the bitwise AND operator, and ~ is the bitwise NOT operator.

3 Programming Flow

This chapter provides the basic guide on how to create component instances and perform encoding with the encoder OpenMAX IL API. For more usage guides, see the OpenMAX Integration Layer Application Programming Interface Specification [1].

3.1 Creating a Component Instance

The following figure shows the process to create a component instance and allocate buffers.

Note: Before creating a component instance, make sure that the component is registered into the OMX Core.

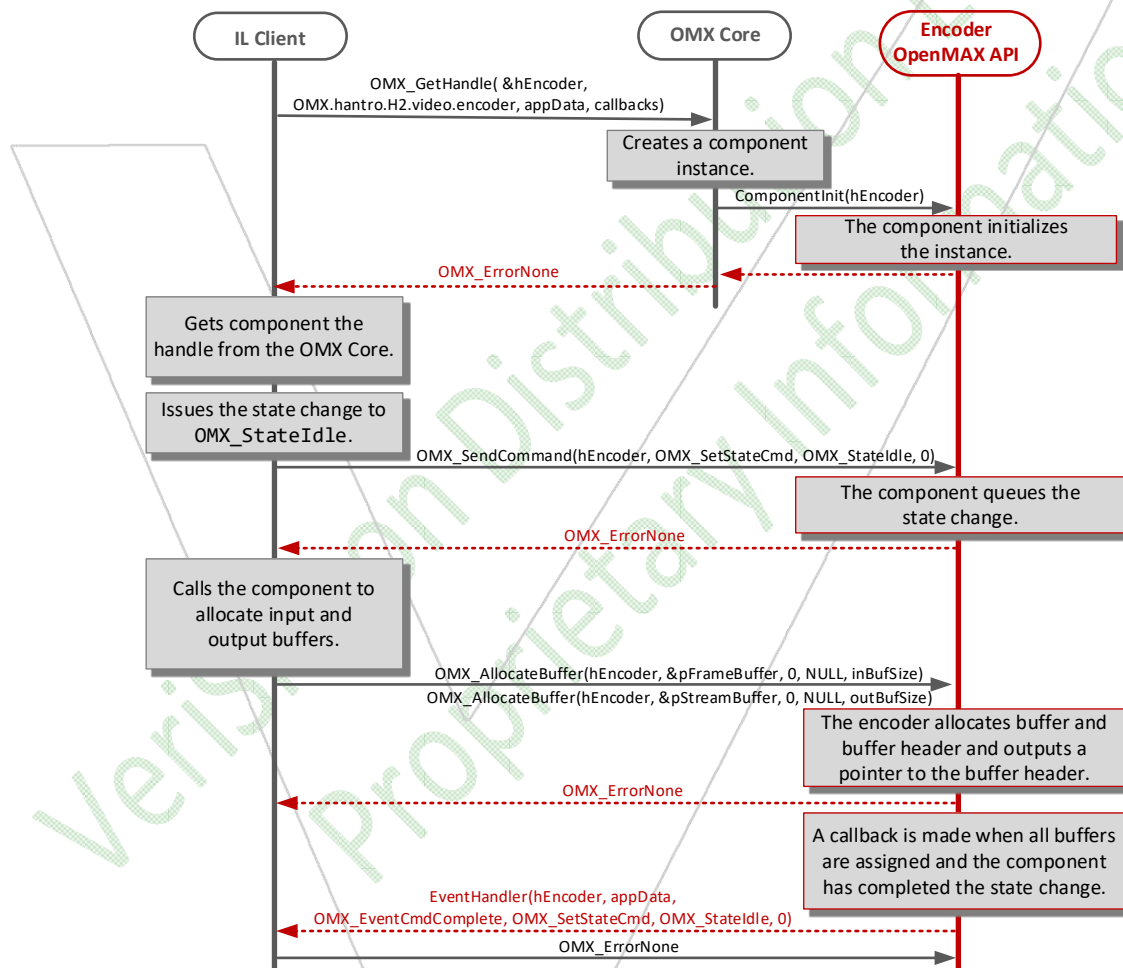


Figure 3. Creating a Component Instance with OpenMAX IL API

3.2 Encoding a Frame

The following figure shows the process to encode a frame with non-tunneled data flow.

Note: Before encoding frames, make sure that a component instance is created and initialized into OMX_StateIdle.

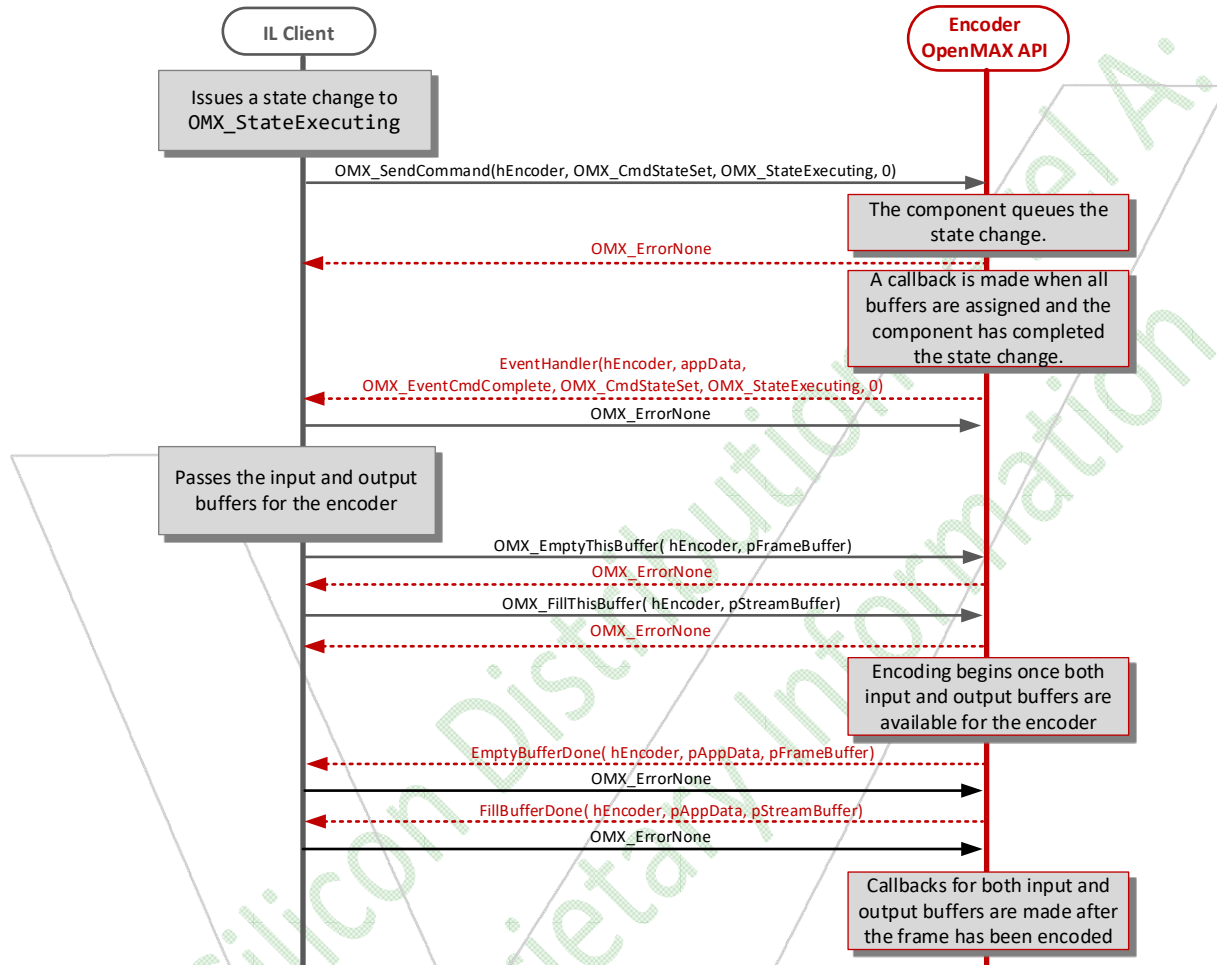


Figure 4. Encoding a Frame with OpenMAX IL API

4 API Components

This chapter describes the VeriSilicon's extension to the OpenMAX IL API, involving buffer flags, enumerations, and data structures. For more information about the OpenMAX IL API, see the OpenMAX Integration Layer Application Programming Interface Specification v1.1.2 [1].

4.1 Buffer Flags

4.1.1 VP8 Base Layer Frame Flag

This flag is set when the buffer contains VP8-encoded base layer frames.

```
#define OMX_BUFFERFLAG_BASE_LAYER    0x00020000
```

4.1.2 VP8 First Layer Frame Flag

This flag is set when the buffer contains VP8-encoded first layer frames.

```
#define OMX_BUFFERFLAG_FIRST_LAYER   0x00040000
```

4.1.3 VP8 Second Layer Frame Flag

This flag is set when the buffer contains VP8-encoded second layer frames.

```
#define OMX_BUFFERFLAG_SECOND_LAYER  0x00080000
```

4.1.4 VP8 Third Layer Frame Flag

This flag is set when the buffer contains VP8-encoded third layer frames.

```
#define OMX_BUFFERFLAG_THIRD_LAYER   0x00100000
```

4.2 Enumerations

4.2.1 OMX_COLOR_FORMATVSITYPE

Defines uncompressed image/video formats.

This enumeration extends OMX_COLOR_FORMATTYPE defined in the OpenMAX IL API.

Enumeration Value	Description
OMX_COLOR_FormatYUV411SemiPlanar	The raster-scan semi-planar YUV411 format.
OMX_COLOR_FormatYUV411PackedSemiPlanar	The raster-scan packed semi-planar YUV411 format.
OMX_COLOR_FormatYUV440SemiPlanar	The raster-scan semi-planar YUV440 format.
OMX_COLOR_FormatYUV440PackedSemiPlanar	The raster-scan packed semi-planar YUV440 format.
OMX_COLOR_FormatYUV444SemiPlanar	The raster-scan semi-planar YUV444 format.
OMX_COLOR_FormatYUV444PackedSemiPlanar	The raster-scan packed semi-planar YUV444 format.
OMX_COLOR_FormatYUV420SemiPlanar4x4Tiled	The 4x4 tiled semi-planar YUV420 format.
OMX_COLOR_FormatYUV420SemiPlanar8x4Tiled	The 8x4 tiled semi-planar YUV420 format.
OMX_COLOR_FormatYUV420SemiPlanarP010	The raster-scan semi-planar YUV420 (P010) format.
OMX_COLOR_FormatYUV420SemiPlanarVU	The raster-scan semi-planar YUV420 (NV21) format.
OMX_COLOR_Format16bitBGR555	The raster-scan 16-bit RGB format.

4.2.2 OMX_INDEXVSITYPE

Defines structure types used when either getting or setting parameters and/or configuration data.

This enumeration extends OMX_INDEXTYPE defined in the OpenMAX IL API.

Enumeration Value	Description
OMX_IndexConfigVideoIntraArea	The OMX_VIDEO_CONFIG_INTRAAREATYPE structure, valid only for H1 encoders.
OMX_IndexConfigVideoRoiArea	The OMX_VIDEO_CONFIG_ROIAREATYPE structure, valid only for H1 encoders.
OMX_IndexConfigVideoRoiDeltaQp	The OMX_VIDEO_CONFIG_ROIDELTAQPTYPE structure, valid only for H1 encoders.

Enumeration Value	Description
OMX_IndexConfigVideoAdaptiveRoi	The OMX_VIDEO_CONFIG_ROIDELTAQPTYPE structure, valid only for H1 encoders.
OMX_IndexConfigVideoVp8TemporalLayers	The OMX_VIDEO_CONFIG_VP8TEMPORALLAYERTYPE structure, valid only for H1 encoders.
OMX_IndexParamVideoHvc	The OMX_VIDEO_PARAM_HEVCTYPE structure.
OMX_IndexConfigVideoIPCMArea	The OMX_VIDEO_CONFIG_IPCMAREATYPE structure.
OMX_IndexParamVideoAvcExt	The OMX_VIDEO_AVCEXTLEVELTYPE structure.

4.2.3 OMX_VIDEO_AVCEXTLEVELTYPE

Defines AVC levels.

Enumeration Value	Description
OMX_VIDEO_AVCLevel52	AVC Level 5.2.
OMX_VIDEO_AVCLevel60	AVC Level 6.0.
OMX_VIDEO_AVCLevel61	AVC Level 6.1.
OMX_VIDEO_AVCLevel62	AVC Level 6.2.

4.2.4 OMX_VIDEO_CODINGVSITYPE

Defines video compression coding types.

This enumeration extends OMX_VIDEO_CODINGTYPE defined in the OpenMAX IL API.

Enumeration Value	Description
OMX_VIDEO_CodingSORENSEN	The Sorenson type.
OMX_VIDEO_CodingDIVX	The DivX type.
OMX_VIDEO_CodingDIVX3	The DivX3 type.
OMX_VIDEO_CodingVP6	The VP6 type.
OMX_VIDEO_CodingAVS	The AVS type.

Enumeration Value	Description
OMX_VIDEO_CodingHEVC	The HEVC (H.265) type.
OMX_VIDEO_CodingVP9	The VP9 type.

4.2.5 OMX_VIDEO_HEVCLEVELTYPE

Defines HEVC (H.265) video levels.

Enumeration Value	Description
OMX_VIDEO_HEVCLLevel1	HEVC (H.265) Level 1.
OMX_VIDEO_HEVCLLevel2	HEVC (H.265) Level 2.
OMX_VIDEO_HEVCLLevel21	HEVC (H.265) Level 2.1.
OMX_VIDEO_HEVCLLevel3	HEVC (H.265) Level 3.
OMX_VIDEO_HEVCLLevel31	HEVC (H.265) Level 3.1.
OMX_VIDEO_HEVCLLevel4	HEVC (H.265) Level 4.
OMX_VIDEO_HEVCLLevel41	HEVC (H.265) Level 4.1.
OMX_VIDEO_HEVCLLevel5	HEVC (H.265) Level 5.
OMX_VIDEO_HEVCLLevel51	HEVC (H.265) Level 5.1.
OMX_VIDEO_HEVCLLevel52	HEVC (H.265) Level 5.2.
OMX_VIDEO_HEVCLLevel6	HEVC (H.265) Level 6.
OMX_VIDEO_HEVCLLevel61	HEVC (H.265) Level 6.1.
OMX_VIDEO_HEVCLLevel62	HEVC (H.265) Level 6.2.
OMX_VIDEO_HEVCLLevelKhronosExtensions	Reserved for Khronos standard extensions.
OMX_VIDEO_HEVCLLevelVendorStartUnused	Reserved for vendor extensions.

4.2.6 OMX_VIDEO_HEVCPROFILETYPE

Defines HEVC (H.265) video profiles.

Enumeration Value	Description
OMX_VIDEO_HEVCProfileMain	HEVC (H.265) Main profile.
OMX_VIDEO_HEVCProfileMain10	HEVC (H.265) Main 10 profile.
OMX_VIDEO_HEVCProfileMainStillPicture	HEVC (H.265) Main Still Picture profile.
OMX_VIDEO_HEVCProfileKhronosExtensions	Reserved for Khronos standard extensions.
OMX_VIDEO_HEVCProfileVendorStartUnused	Reserved for vendor extensions.

4.2.7 OMX_VIDEO_VP8LEVELTYPE

Defines VP8 video versions.

Enumeration Value	Description
OMX_VIDEO_VP8Level_Version0	VP8 version 0.
OMX_VIDEO_VP8Level_Version1	VP8 version 1.
OMX_VIDEO_VP8Level_Version2	VP8 version 2.
OMX_VIDEO_VP8Level_Version3	VP8 version 3.
OMX_VIDEO_VP8LevelUnknown	Unknown VP8 version.

4.2.8 OMX_VIDEO_VP8PROFILETYPE

Defines VP8 video profiles.

Enumeration Value	Description
OMX_VIDEO_VP8ProfileMain	VP8 Main profile.
OMX_VIDEO_VP8ProfileUnknown	Unknown VP8 profile.

4.3 Structures

4.3.1 OMX_CONFIG_INTRAREFRESHVOPTYPE

The structure that defines the parameter for I-VOP.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
IntraRefreshVOP	OMX_BOOL	Indicates whether to encode the next frame as an I-VOP.

4.3.2 OMX_CONFIG_FRAMERATETYPE

The structure that defines the parameter for frame rate configuration.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
xEncodeFramerate	OMX_U32	The encoding frame rate, in Q16 format.

4.3.3 OMX_PARAM_COMPONENTROLETYPE

The structure that contains information about component roles.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
cRole	OMX_U32	The role of each standard component.

4.3.4 OMX_VIDEO_CONFIG_ADAPTIVEROI

The structure used to configure an adaptive ROI.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
nAdaptiveROI	OMX_S32	The QP delta value for the adaptive ROI.
nAdaptiveROIColor	OMX_S32	The color temperature for the adaptive ROI. Valid values: -10: 2000K. 0: 3000K. 10: 5000K.

4.3.5 OMX_VIDEO_CONFIG_AVCINTRAPERIOD

The structure that defines parameters for AVC IDR period configuration.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
nIDRPeriod	OMX_U32	(Reserved) The interval between IDR frames.
nPFrames	OMX_U32	The number of P-frames between consecutive I-frames.

4.3.6 OMX_VIDEO_CONFIG_INTRAAREATYPE

The structure used to configure an intra area.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.

Member	Data Type	Description
nPortIndex	OMX_U32	The port to set.
bEnable	OMX_BOOL	Indicates whether to enable the intra area.
nTop	OMX_U32	The top MB or CTB row inside the area. Valid values: [0, heightMbs - 1], where heightMbs indicates the picture height in the unit of MB or CTB
nLeft	OMX_U32	The leftmost MB or CTB column inside the area. Valid values: [0, widthMbs - 1], where widthMbs indicates the picture width in the unit of MB or CTB
nBottom	OMX_U32	The bottom MB or CTB row inside the area. Valid values: [nTop, heightMbs - 1], where heightMbs indicates the picture height in the unit of MB or CTB
nRight	OMX_U32	The rightmost MB or CTB column inside the area. Valid values: [nLeft, widthMbs - 1], where widthMbs indicates the picture width in the unit of MB or CTB

4.3.7 OMX_VIDEO_CONFIG_IPCMAREATYPE

The structure used to configure an IPCM area.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
bEnable	OMX_BOOL	Indicates whether to enable the IPCM area.
nArea	OMX_U32	The ID of the IPCM area. Valid values: [1, 2]
nTop	OMX_U32	The top MB or CTB row inside the area. Valid values: [0, heightMbs - 1], where heightMbs indicates the picture height in the unit of MB or CTB

Member	Data Type	Description
nLeft	OMX_U32	The leftmost MB or CTB column inside the area. Valid values: [0, widthMbs - 1], where widthMbs indicates the picture width in the unit of MB or CTB
nBottom	OMX_U32	The bottom MB or CTB row inside the area. Valid values: [nTop, heightMbs - 1], where heightMbs indicates the picture height in the unit of MB or CTB
nRight	OMX_U32	The rightmost MB or CTB column inside the area. Valid values: [nLeft, widthMbs - 1], where widthMbs indicates the picture width in the unit of MB or CTB

4.3.8 OMX_VIDEO_CONFIG_ROIAREATYPE

The structure used to configure a rectangular ROI.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
bEnable	OMX_BOOL	Indicates whether to enable the rectangular ROI.
nArea	OMX_U32	The ID of the ROI. Valid values: [1, 2]
nTop	OMX_U32	The top MB or CTB row inside the ROI. Valid values: [0, heightMbs - 1], where heightMbs indicates the picture height in the unit of MB or CTB
nLeft	OMX_U32	The leftmost MB or CTB column inside the ROI. Valid values: [0, widthMbs - 1], where widthMbs indicates the picture width in the unit of MB or CTB
nBottom	OMX_U32	The bottom MB or CTB row inside the ROI. Valid values: [nTop, heightMbs - 1], where heightMbs indicates the picture height in the unit of MB or CTB

Member	Data Type	Description
nRight	OMX_U32	The rightmost MB or CTB column inside the ROI. Valid values: [nLeft, widthMbs - 1], where widthMbs indicates the picture width in the unit of MB or CTB

4.3.9 OMX_VIDEO_CONFIG_ROIDELTAQPTYPE

The structure used to configure the QP delta for an ROI.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
nArea	OMX_U32	The ID of the ROI. Valid values: [1, 2]
nDeltaQP	OMX_S32	The QP delta value for the ROI. Valid values: [-127, 0] for VP8 and [-15, 0] for H.264

4.3.10 OMX_VIDEO_CONFIG_VP8TEMPORALLAYERTYPE

The structure used to configure VP8 temporal layers.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
nBaseLayerBitrate	OMX_U32	The bits per second at the base layer. Valid values: [10000, 40000000]
nLayer1Bitrate	OMX_U32	The bits per second at the first layer. Valid values: [10000, 40000000]

Member	Data Type	Description
nLayer2Bitrate	OMX_U32	The bits per second at the second layer. Valid values: [10000, 40000000]
nLayer3Bitrate	OMX_U32	The bits per second at the third layer. Valid values: [10000, 40000000]

4.3.11 OMX_VIDEO_PARAM_AVCEXTTYPE

The structure that defines parameters for the AVC video standard.

This structure extends OMX_VIDEO_PARAM_AVCTYPE defined in the OpenMAX IL API.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
nBitDepthLuma	OMX_U32	The valid bit depth of the luma component.
nBitDepthChroma	OMX_U32	The valid bit depth of the chroma components.
gopSize	OMX_U32	The size of the GOP structure. Valid values: [0, 8], where value 0 indicates an adaptive GOP size
hrdCpbSize	OMX_U32	The size of the coded picture buffer (CPB) in bits. When HRD is enabled, each encoded frame cannot be larger than the specified size.
firstPic	OMX_S32	The first frame to be encoded.
lastPic	OMX_S32	The last frame to be encoded.

4.3.12 OMX_VIDEO_PARAM_BITRATETYPE

The structure that defines parameters for bit rate configuration.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
eControlRate	OMX_VIDEO_CONTROLRATETYPE	The rate control mode.
nTargetBitrate	OMX_U32	The target bit rate, in bit per seconds.

4.3.13 OMX_VIDEO_PARAM_HEVCTYPE

The structure that defines parameters for the HEVC (H.265) video standard.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
eProfile	OMX_VIDEO_HEVCPROFILETYPE	The profile of the stream.
eLevel	OMX_VIDEO_HEVCLEVELTYPE	The level of the stream.
nPFrames	OMX_U32	The number of P-frames between consecutive I-frames.
nBFrames	OMX_U32	The number of P-frames between consecutive I-frames.
exp_of_input_alignment	OMX_U32	The alignment requirement for both luma and chroma input frame buffers. The start address of each line in the buffer must be aligned to power(2, exp_of_input_alignment). Valid values: [4, 12]

Member	Data Type	Description
fieldOrder	OMX_U32	The field order for interlaced coding. Valid values: 0: bottom field first. 1: top field first.
nBitDepthLuma	OMX_U32	The valid bit depth of the luma component.
nBitDepthChroma	OMX_U32	The valid bit depth of the chroma components.
byteStream	OMX_U32	The HEVC (H.265) stream type. Valid values: 0: NAL unit stream. 1: byte stream.
videoRange	OMX_S32	The video signal sample range in the encoded stream. Valid values: 0: Y in range [16, 235]; Cb and Cr in range [16, 240]. 1: Y, Cb, and Cr in range [0, 255].
enableCabac	OMX_S32	The entropy coding mode. Valid values: 0: content-adaptive variable-length coding (CAVLC). 1: content-based adaptive binary arithmetic coding (CABAC).
bCabacInitFlag	OMX_BOOL	The CABAC table initialization flag cabac_init_flag.

Member	Data Type	Description
sliceSize	OMX_S32	The number of CTB or MB rows in each slice. If the value is 0, slice encoding is disabled and the entire frame is encoded as a slice. If the value is not 0, a total of $\lceil (\text{height} + \text{ctu_size} - 1) / \text{ctu_size} \rceil$ slices are created, with the height of each slice equal to $(\text{ctu_size} \times \text{sliceSize})$ except the last slice.
bEnableSAO	OMX_BOOL	Indicates whether sample adaptive offset (SAO) is enabled.
nTcOffset	OMX_S32	The de-blocking parameter <code>tc_offset</code>. Valid values: [-6, 6]
nBetaOffset	OMX_S32	The de-blocking parameter <code>beta_offset</code>. Valid values: [-6, 6]
bEnableDeblockOverride	OMX_BOOL	Indicates whether de-blocking override is enabled between frames. This member is valid only for HEVC (H.265).
bDeblockOverride	OMX_BOOL	Indicates whether de-blocking override is enabled between slices. This member is valid only for HEVC (H.265).
num_tile_columns	OMX_U32	(For VC8000E/VC9000E only) The number of tile columns to divide the picture.
num_tile_rows	OMX_U32	(For VC8000E/VC9000E only) The number of tile rows to divide the picture.

Member	Data Type	Description
loop_filter_across_tiles_enabled_flag	OMX_U32	(For VC8000E/VC9000E only) Indicates whether to enable the loop filter across the tile boundary. Valid values: 0: disable. 1: enable.
bEnableScalingList	OMX_BOOL	Indicates whether the default scaling list is used.
RpsInSliceHeader	OMX_U32	Indicates whether to specify the RPS in the slice header instead of using the pre-defined RPS from the SPS. Valid values: 0: use the pre-defined RPS from the SPS. 1: specify the RPS in the slice header.
intraPicRate	OMX_U32	The intra-picture rate, which is the interval between two IDR frames.
tolMovingBitRate	OMX_S32	The tolerance percentage of the maximum moving bit rate over the target bit rate. Valid values: [0, 2000]
monitorFrames	OMX_S32	The number of frames to be monitored for calculating the moving bit rate. Valid values: [10, 120]
bitVarRangel	OMX_S32	The permitting variance percentage over average bits per I frame from the target bit rate. Valid values: [10, 10000] for VC8000E/VC9000E [10, 2000] for H2 V4.1

Member	Data Type	Description
bitVarRangeP	OMX_S32	The permitting variance percentage over average bits per P frame from the target bit rate. Valid values: [10, 10000] for VC8000E/VC9000E [10, 2000] for H2 V4.1
bitVarRangeB	OMX_S32	The permitting variance percentage over average bits per B frame from the target bit rate. Valid values: [10, 10000] for VC8000E/VC9000E [10, 2000] for H2 V4.1
staticScenelbitPercent	OMX_S32	(For VC8000E/VC9000E only) The percentage of I frame bits in static scenes. Valid values: [0, 100]
smoothPsnrInGOP	OMX_S32	(For VC8000E/VC9000E only) Indicates whether to enable smooth PSNR for frames in one GOP. Valid values: 0: disable. 1: enable.
ctbRc	OMX_S32	The block-level rate control mode for adjusting QP inside a frame. Valid values: [0, 3] for VC8000E/VC9000E [0, 1] for H2
blockRCSIZE	OMX_S32	The block size for block-level rate control. Valid values: 0: block size 64x64. 1: block size 32x32. 2: block size 16x16.

Member	Data Type	Description
rcQpDeltaRange	OMX_S32	(For VC8000E/VC9000E only) The maximum absolute delta between block-level QP value and picture-level QP value.
rcBaseMBComplexity	OMX_S32	(For VC8000E/VC9000E only) The complexity threshold for subjective block-level rate control. Valid values: [0, 31]
tolCtbRcInter	OMX_S32	(For VC8000E/VC9000E only) The tolerance of block-level rate control for inter frames.
tolCtbRcIntra	OMX_S32	(For VC8000E/VC9000E only) The tolerance of block-level rate control for intra frames.
ctbRowQpStep	OMX_S32	(For VC8000E/VC9000E only) The maximum accumulated QP adjustment step per CTB row allowed by block-level rate control.
picQpDeltaMin	OMX_S32	(For VC8000E/VC9000E only) The minimum QP delta in picture-level rate control.
picQpDeltaMax	OMX_S32	(For VC8000E/VC9000E only) The maximum QP delta in picture-level rate control.
hrdCpbSize	OMX_U32	The size of the coded picture buffer (CPB) in bits. When HRD is enabled, each encoded frame cannot be larger than the specified size.

Member	Data Type	Description
bitrateWindow	OMX_S32	(For VC8000E/VC9000E only) The number of frames within which the rate control algorithm tries to achieve the target bit rate. Valid values: [1, 300]
gopSize	OMX_U32	The size of the GOP structure. Valid values: [0, 8], where value 0 indicates an adaptive GOP size.
ltrInterval	OMX_S32	The long-term reference (LTR) interval.
longTermGapOffset	OMX_S32	The first frame after LTR that uses the LTR as reference.
longTermGap	OMX_S32	The POC delta between two consecutive frames that use the same LTR as reference.
longTermQpDelta	OMX_S32	The QP delta for frames using LTR.
gopLowdelay	OMX_U32	Indicates whether to use low-delay GOP configuration. Valid values: 0: do not use. 1: use.
picSkip	OMX_S32	Indicates whether to enable picture skipping for rate control. This member is invalid for B-frame encoding.
intraQpDelta	OMX_S32	The delta between QP and intra QP values. Valid values: [-51, 51]
fixedIntraQp	OMX_S32	The fixed QP value for all intra frames. Valid values: [0, 51], where value 0 indicates that the encoder automatically calculates the intra QP

Member	Data Type	Description
chromaQpOffset	OMX_U32	The chroma QP offset. Valid values: [-12, 12]
vbr	OMX_S32	(For VC8000E/VC9000E only) Indicates whether to enable variable bit rate control based on the minimum QP allowed. Valid values: 0: disable. 1: enable.
gdrDuration	OMX_U32	The duration it takes to perform gradual decoder refresh (GDR), in the unit of frame. Value 0 indicates to disable GDR.
cirStart	OMX_U32	The order count of the first MB or CTB for cyclic intra refresh (CIR).
cirInterval	OMX_U32	The interval for CIR, in MB or CTB.
ipcmMapEnable	OMX_U32	Whether to enable the IPCM map. Valid values: 0: disable. 1: enable.
roiQp[8]	OMX_S32	(For VC8000E/VC9000E only) The absolute QP value for ROI n . Valid values: [0, 51]
roiMapDeltaQpBlockUnit	OMX_U32	The size of the unit block for the QP map. Valid values: 0: 64x64. 1: 32x32. 2: 16x16. 3: 8x8.

Member	Data Type	Description
roiMapDeltaQpEnable	OMX_U32	Whether to enable the ROI map for forcing QP delta or absolute QP values. Valid values: 0: disable. 1: enable.
RoiQpDelta_ver	OMX_U32	(For VC8000E/VC9000E only) The version of the QP information format used in the QP map. Valid values: [1, 4]
rdoLevel	OMX_U32	(For VC8000E/VC9000E only) The RDO level, which controls how much effort the video encoder uses for mode selection. Valid values: [1, 3]
enableRdoQuant	OMX_S32	(For VC8000E/VC9000E only) Indicates whether to enable rate-distortion optimized quantization (RDOQ). Valid values: 0: disable. 1: enable.
ssim	OMX_U32	(For VC8000E/VC9000E only) Indicates whether to enable SSIM calculation. Valid values: 0: disable. 1: enable.
cuInfoVersion	OMX_S32	(For VC8000E/VC9000E only) The CU information version.

Member	Data Type	Description
vui_timing_info_enable	OMX_S32	(For VC8000E/VC9000E only) Indicates whether to write VUI timing information in SPS. Valid values: 0: do not write. 1: write.
hashtype	OMX_S32	(For VC8000E/VC9000E only) The method to calculate the hash value of the output stream. Valid values: 0: hash value calculation disabled. 1: standard CRC32. 2: 32-bit checksum.
streamMultiSegmentMode	OMX_U32	(For VC8000E/VC9000E only) The multi-segment mode of output stream buffers. Valid values: [0, 2]
streamMultiSegmentAmount	OMX_U32	(For VC8000E/VC9000E only) The total amount of segments to control loopback, software handshaking, and IRQ.
noiseReductionEnable	OMX_U32	Indicates whether to enable noise reduction. Valid values: 0: disable. 1: enable.
noiseLow	OMX_U32	The minimum noise value. Valid values: [1, 30]
noiseFirstFrameSigma	OMX_U32	The noise estimation for start frames. Valid values: [1, 30]

Member	Data Type	Description
hdr10_display_enable	OMX_U32	(For VC8000E/VC9000E only) Indicates whether to insert mastering display color volume information. Valid values: 0: do not insert. 1: insert.
hdr10_dx0	OMX_U32	(For VC8000E/VC9000E only) The normalized X chromaticity coordinate of component 0.
hdr10_dy0	OMX_U32	(For VC8000E/VC9000E only) The normalized Y chromaticity coordinate of component 0.
hdr10_dx1	OMX_U32	(For VC8000E/VC9000E only) The normalized X chromaticity coordinate of component 1.
hdr10_dy1	OMX_U32	(For VC8000E/VC9000E only) The normalized Y chromaticity coordinate of component 1.
hdr10_dx2	OMX_U32	(For VC8000E/VC9000E only) The normalized X chromaticity coordinate of component 2.
hdr10_dy2	OMX_U32	(For VC8000E/VC9000E only) The normalized Y chromaticity coordinate of component 2.
hdr10_wx	OMX_U32	(For VC8000E/VC9000E only) The normalized X chromaticity coordinate of the white point.
hdr10_wy	OMX_U32	(For VC8000E/VC9000E only) The normalized Y chromaticity coordinate of the white point.

Member	Data Type	Description
hdr10_maxluma	OMX_U32	(For VC8000E/VC9000E only) The nominal maximum display luminance.
hdr10_minluma	OMX_U32	(For VC8000E/VC9000E only) The nominal minimum display luminance.
hdr10_lightlevel_enable	OMX_U32	(For VC8000E/VC9000E only) Indicates whether to insert content light level information. Valid values: 0: do not insert. 1: insert.
hdr10_maxlight	OMX_U32	(For VC8000E/VC9000E only) The maximum content light level.
hdr10_avglight	OMX_U32	(For VC8000E/VC9000E only) The picture average light level.
hdr10_color_enable	OMX_U32	(For VC8000E/VC9000E only) Indicates whether to insert color description. Valid values: 0: do not insert. 1: insert.
hdr10_primary	OMX_U32	(For VC8000E/VC9000E only) The index of chromaticity coordinates. For details, see Table E.3 in the HEVC specification.

Member	Data Type	Description
hdr10_transfer	OMX_U32	(For VC8000E/VC9000E only) The reference opto-electronic transfer characteristic function of the source picture. For details, see Table E.4 in the HEVC specification.
hdr10_matrix	OMX_U32	(For VC8000E/VC9000E only) The index of matrix coefficients used in deriving luma and chroma signals from the green, blue, and red or Y, Z, and X primaries. For details, see Table E.5 in the HEVC specification.
firstPic	OMX_S32	The first frame to be encoded.
lastPic	OMX_S32	The last frame to be encoded.
nRefFrames	OMX_U32	(For H2 only) The maximum number of reference frames to use for inter motion search. Valid values: (1, 16)
bStrongIntraSmoothing	OMX_BOOL	Indicates whether to enable the HEVC (H.265) IntraTU32x32 strong intra smoothing filter. Valid values: 0: disable. 1: enable.
pcm_loop_filter_disabled_flag	OMX_U32	Indicates whether to disable the deblocking filter for IPCM. Valid values: 0: enable. 1: disable.

4.3.14 OMX_VIDEO_PARAM_INTRAREFRESHTYPE

The structure that defines parameters for intra refresh mode configuration.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
eRefreshMode	OMX_VIDEO_INTRAREFRESHTYPE	The intra refresh mode.
nAirMBs	OMX_U32	The number of intra macroblocks to refresh in a frame when AIR is enabled.
nAirRef	OMX_U32	The number of times a motion-marked macroblock has to be intra coded.
nCirMBs	OMX_U32	The number of consecutive macroblocks to be coded as intra when CIR is enabled.

4.3.15 OMX_VIDEO_PARAM_PROFILELEVELTYPE

The structure that contains the information about the video profile and level.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
eProfile	OMX_U32	The profile of the stream. The data type is OMX_VIDEO_AVCPROFILETYPE, OMX_VIDEO_H263PROFILETYPE, or OMX_VIDEO_MPEG4PROFILETYPE.
eLevel	OMX_U32	The version of the stream. The data type is OMX_VIDEO_AVCLEVELTYPE, OMX_VIDEO_H263LEVELTYPE, or OMX_VIDEO_MPEG4LEVELTYPE.
nProfileIndex	OMX_U32	The index of the profile.

4.3.16 OMX_VIDEO_PARAM_VP8TYPE

The structure that defines parameters for the VP8 video standard.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
eProfile	OMX_VIDEO_VP8PROFILETYPE	The profile of the stream.
eLevel	OMX_VIDEO_VP8LEVELTYPE	The version of the stream.
nDCTPartitions	OMX_U32	The number of DCT coefficient data partitions.
bErrorResilientMode	OMX_BOOL	Indicates whether to enable the error resilient mode.

4.3.17 OMX_VIDEO_VP8REFERENCEFRAMETYPE

The structure that defines parameters for VP8 reference frame configuration.

Member	Data Type	Description
nSize	OMX_U32	The size of the structure in bytes.
nVersion	OMX_VERSIONTYPE	The OMX specification version information.
nPortIndex	OMX_U32	The port to set.
bPreviousFrameRefresh	OMX_BOOL	Indicates whether to use the next frame for refreshing the immediately previous frame.
bGoldenFrameRefresh	OMX_BOOL	Indicates whether to encode the next frame as a golden reference frame.
bAlternateFrameRefresh	OMX_BOOL	Indicates whether to encode the next frame as an alternate reference frame.
bUsePreviousFrame	OMX_BOOL	Indicates whether to use the immediately previous frame for prediction.

Member	Data Type	Description
bUseGoldenFrame	OMX_BOOL	Indicates whether to use the golden reference frame for prediction.
bUseAlternateFrame	OMX_BOOL	Indicates whether to use the alternate reference frame for prediction.

4.3.18 OUTPUT_BUFFER_PRIVATE

The structure that contains the information of a buffer that is returned to the IL client with the FillBufferDone() callback. This structure can be accessed through the pOutputPortPrivate pointer in the OMX_BUFFERHEADERTYPE structure.

Separate luminance and chrominance buffers are required when VP9 encoder is used in tiled mode.

Member	Data Type	Description
pLumaBase	OMX_U8*	The virtual address of the luminance buffer.
nLumaBusAddress	OMX_U64	The physical address of the luminance buffer.
nLumaSize	OMX_U32	The size of the luminance data.
pChromaBase	OMX_U8*	The virtual address of the chrominance buffer.
nChromaBusAddress	OMX_U64	The physical address of the chrominance buffer.
nChromaSize	OMX_U32	The size of the chrominance data.
nBitDepthLuma	OMX_U32	The valid bit depth of the luma component.
nBitDepthChroma	OMX_U32	The valid bit depth of the chroma components.
nFrameWidth	OMX_U32	The picture width in pixels.
nFrameHeight	OMX_U32	The picture height in pixels.
nStride	OMX_U32	The picture stride in bytes.
nPicId[2]	OMX_U32	The identifier of each picture in encoding order. For H.264 interlaced stream, nPicId[0] and nPicId[1] are used to identify top and bottom fields.
realloc	OMX_BOOL	Indicates whether to re-allocate the output buffer.
singleField	OMX_BOOL	Indicates whether the output buffer contains only one field.

5 API Functions

This chapter describes the OpenMAX IL API functions supported for VeriSilicon's Hantro encoders. For more information about the OpenMAX IL API, see the OpenMAX Integration Layer Application Programming Interface Specification v1.1.2 [1].

Note: The Initialization function is called from the OMX Core when creating a component instance and after that the OMX defined macros (for example OMX_GetComponentVersion) should be used for invoking all the other functions.

5.1 HantroHwEncOmx_encoder_init()

Description

This function allocates and initializes the private structure of a component instance.

The application must allocate an OMX_COMPONENTTYPE structure before calling this function. After this function is called, the structure is then maintained by the OMX Core.

Syntax

```
OMX_ERRORTYPE HantroHwEncOmx_encoder_init(  
    OMX_HANDLETYPE hComponent  
);
```

Parameters

IN/OUT	hComponent	The handle of the component to be initialized.
--------	------------	--

Returns

OMX_ERRORTYPE

5.2 GetComponentVersion()

Description

This function returns the version information about the component.

Syntax

```
OMX_ERRORTYPE GetComponentVersion(
    OMX_HANDLETYPE hComponent,
    OMX_STRING pComponentName,
    OMX_VERSIONTYPE *pComponentVersion,
    OMX_VERSIONTYPE *pSpecVersion,
    OMX_UUIDTYPE *pComponentUUID
);
```

Parameters

IN	hComponent	The handle of the component to be queried.
OUT	pComponentName	The name of the queried component.
OUT	*pComponentVersion	A pointer to the information about the version of the queried component.
OUT	*pSpecVersion	A pointer to the information about the OMX version against which the queried component is built.
OUT	*pComponentUUID	A pointer to the universal unique identifier (UUID) of the queried component.

Returns

OMX_ERRORTYPE

5.3 SendCommand()

Description

This function sends a command to a component.

A call to this function is a non-blocking call. After this function is called, the component checks the parameters and then queues the specified command to the to-be-executed command thread of the component. At the end of the command execution, the component triggers the EventHandler() callback.

Syntax

```
OMX_ERRORTYPE SendCommand(
    OMX_HANDLETYPE hComponent,
    OMX_COMMANDTYPE Cmd,
    OMX_U32 nParam1,
    OMX_PTR pCmdData
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	Cmd	The command to be executed. For the list of supported commands, see Section 5.3.1, Supported Commands .
IN	nParam1	The integer parameter for the command to be executed.
IN	pCmdData	A pointer to the command-specific data that cannot be represented using the integer parameter nParam1.

Returns

OMX_ERRORTYPE

5.3.1 Supported Commands

The following commands are supported for the [SendCommand\(\)](#) function:

- OMX_CommandStateSet
 - OMX_StateLoaded
 - OMX_StateIdle
 - OMX_StateExecuting
 - OMX_StatePause
 - OMX_StateInvalid
- OMX_CommandFlush
- OMX_CommandPortDisable
- OMX_CommandPortEnable
- OMX_CommandMarkBuffer

5.4 GetParameter()

Description

This function returns a set of current parameter settings from a component.

The nParamIndex parameter indicates the type of the structure being requested from the component.

- The application must allocate a corresponding structure and fill in the structure size and version information before calling this function.
- When the parameter settings in the structure apply to a specific port, the application must fill in the appropriate nPortIndex value in the structure.
- If the component has no parameter settings changed in the structure, the component returns the structure with default settings.

Some parameters may be invalid if they are not supported by the encoder hardware. For details, see [Chapter 2, Feature Support](#).

Syntax

```
OMX_ERRORTYPE GetParameter(
    OMX_HANDLETYPE hComponent,
    OMX_INDEXTYPE nParamIndex,
    OMX_PTR        ComponentParameterStructure
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	nParamIndex	The index of the structure type of ComponentParameterStructure. For the list of supported structure types, see Section 5.4.1, Supported Structures .
IN/OUT	ComponentParameterStructure	A pointer to the structure that the component fills in.

Returns

OMX_ERRORTYPE

5.4.1 Supported Structures

The following tables list the structures that are supported for `GetParameter()` and their corresponding `nParamIndex` values in video and image domains, respectively.

Table 2. Structures Supported for `GetParameter()` in Video Domain

nParamIndex Value	Structure
OMX_IndexParamVideoInit	OMX_PORT_PARAM_TYPE
OMX_IndexParamCompBufferSupplier	OMX_PARAM_BUFFERSUPPLIERTYPE
OMX_IndexParamPortDefinition	OMX_PARAM_PORTDEFINITIONTYPE
OMX_IndexParamVideoPortFormat	OMX_VIDEO_PARAM_PORTFORMATTYPE
OMX_IndexParamVideoAvc	OMX_VIDEO_PARAM_AVCTYPE
OMX_IndexParamVp8	OMX_VIDEO_PARAM_VP8TYPE
OMX_IndexParamVideoH263	OMX_VIDEO_PARAM_H263TYPE
OMX_IndexParamVideoMpeg4	OMX_VIDEO_PARAM_MPEG4TYPE
OMX_IndexParamVideoHvc	OMX_VIDEO_PARAM_HEVCTYPE
OMX_IndexParamVideoAvcExt	OMX_VIDEO_PARAM_AVCEXTTYPE
OMX_IndexParamVideoErrorCorrection	OMX_VIDEO_PARAM_ERRORCORRECTIONTYPE
OMX_IndexParamVideoProfileLevelCurrent	OMX_VIDEO_PARAM_PROFILELEVELTYPE
OMX_IndexParamVideoProfileLevelQuerySupported	OMX_VIDEO_PARAM_PROFILELEVELTYPE
OMX_IndexParamCommonDeblocking	OMX_PARAM_DEBLOCKINGTYPE
OMX_IndexParamVideoQuantization	OMX_VIDEO_PARAM_QUANTIZATIONTYPE
OMX_IndexParamVideoBitrate	OMX_VIDEO_PARAM_BITRATETYPE
OMX_IndexParamStandardComponentRole	OMX_PARAM_COMPONENTROLETYPE
OMX_IndexParamPriorityMgmt	OMX_PRIORITYMGMTTYPE
OMX_IndexParamVideoIntraRefresh	OMX_VIDEO_PARAM_INTRAREFRESHTYPE

Table 3. Structures Supported for GetParameter() in Image Domain

nParamIndex Value	Structure
OMX_IndexParamImageInit	OMX_PORT_PARAM_TYPE
OMX_IndexParamCompBufferSupplier	OMX_PARAM_BUFFERSUPPLIERTYPE
OMX_IndexParamPortDefinition	OMX_PARAM_PORTDEFINITIONTYPE
OMX_IndexParamImagePortFormat	OMX_IMAGE_PARAM_PORTFORMATTYPE
OMX_IndexParamQFactor	OMX_IMAGE_PARAM_QFACTORTYPE
OMX_IndexParamStandardComponentRole	OMX_PARAM_COMPONENTROLETYPE
OMX_IndexParamPriorityMgmt	OMX_PRIORITYMGMTTYPE

5.5 SetParameter()

Description

This function sends the parameter settings in a structure to a component.

You can call this function once to send the parameter settings in each structure. This function can only be called when the component is in the OMX_LoadedState state, or when the port is disabled if the parameter settings apply to a specific port.

The nIndex parameter indicates the type of the structure being sent to the component.

- The application must allocate a corresponding structure and fill in the structure size and version information as well as other parameter settings before calling this function.
- The application is free to dispose of the structure after the function call, as the component has copied any data it retains.

Some parameters may be invalid if they are not supported by the encoder hardware. For details, see [Chapter 2, Feature Support](#).

Syntax

```
OMX_ERRORTYPE SetParameter(
    OMX_HANDLETYPE hComponent,
    OMX_INDEXTYPE nIndex,
    OMX_PTR ComponentParameterStructure
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	nIndex	The index of the structure type of ComponentParameterStructure. For the list of supported structure types, see Section 5.5.1, Supported Structures .
IN/OUT	ComponentParameterStructure	A pointer to the structure to be sent to the component.

Returns

OMX_ERRORTYPE

5.5.1 Supported Structures

The following tables list the structures that are supported for SetParameter() and their corresponding nIndex values in video and image domains, respectively.

Table 4. Structures Supported for GetParameter() in Video Domain

nIndex Value	Structure
OMX_IndexParamVideoInit	OMX_PORT_PARAM_TYPE
OMX_IndexParamCompBufferSupplier	OMX_PARAM_BUFFERSUPPLIERTYPE

nIndex Value	Structure
OMX_IndexParamPortDefinition	OMX_PARAM_PORTDEFINITIONTYPE Note 1: The following structure members are extended for configuration through port 0: • xFrameRate: indicates the frame rate of the encoder. • nFrameWidth: indicates the width of input frames. • nFrameHeight: indicates the height of input frames. • nStride: indicates the number of bytes per span of input frames. • eColorFormat: indicates the color format of input frames. Note 2: The following structure members are extended for configuration through port 1: • xFrameRate: indicates the frame rate of the encoder. • nBitRate: indicates the target bit rate in bits per second. • nFrameWidth and nFrameHeight: indicate the resolution of encoded frames. • nSliceHeight: indicates the height of each encoded slice in pixels. • eCompressionFormat: indicates the compression format of output frames.
OMX_IndexParamVideoPortFormat	OMX_VIDEO_PARAM_PORTFORMATTYPE
OMX_IndexParamVideoAvc	OMX_VIDEO_PARAM_AVCTYPE Note: This structure is configurable only through port 1.
OMX_IndexParamVideoVp8	OMX_VIDEO_PARAM_VP8TYPE Note: This structure is configurable only through port 1.
OMX_IndexParamVideoHvc	OMX_VIDEO_PARAM_HEVCTYPE Note: This structure is configurable only through port 1.
OMX_IndexParamVideoAvcExt	OMX_VIDEO_PARAM_AVCEXTTYPE Note: This structure is configurable only through port 1.
OMX_IndexParamCommonDeblocking	OMX_PARAM_DEBLOCKINGTYPE Note: This structure is valid only for H.264 and HEVC (H.265) and configurable only through port 1.
OMX_IndexParamVideoBitrate	OMX_VIDEO_PARAM_BITRATETYPE Note: This structure is configurable only through port 1.

nIndex Value	Structure
OMX_IndexParamVideoErrorCorrection	OMX_VIDEO_PARAM_ERRORCORRECTIONTYPE Note: This structure is configurable only through port 1.
OMX_IndexParamVideoQuantization	OMX_VIDEO_PARAM_QUANTIZATIONTYPE Note: This structure is configurable only through port 1.
OMX_IndexParamStandardComponentRole	OMX_PARAM_COMPONENTROLETYPE
OMX_IndexParamPriorityMgmt	OMX_PRIORITYMGMTTYPE
OMX_IndexParamVideoProfileLevelCurrent	OMX_VIDEO_PARAM_PROFILELEVELTYPE

Table 5. Structures Supported for SetParameter() in Image Domain

nIndex Value	Structure
OMX_IndexParamImageInit	OMX_PORT_PARAM_TYPE
OMX_IndexParamCompBufferSupplier	OMX_PARAM_BUFFERSUPPLIERTYPE
OMX_IndexParamPortDefinition	OMX_PARAM_PORTDEFINITIONTYPE Note 1: The following structure members are extended for configuration through port 0: - nFrameWidth: indicates the width of input frames. - nFrameHeight: indicates the height of input frames. - nStride: indicates the number of bytes per span of input frames. - nSliceHeight: indicates the height of each slice in pixels. Value 0 indicates not to divide input frames into slices. - eColorFormat: indicates the color format of input frames. Note 2: Structure members nFrameWidth and nFrameHeight are extended for configuration through port 1. They indicate the resolution of encoded frames.
OMX_IndexParamImagePortFormat	OMX_IMAGE_PARAM_PORTFORMATTYPE
OMX_IndexParamQFactor	OMX_IMAGE_PARAM_QFACTORTYPE Note: This structure is configurable only through port 1.
OMX_IndexParamPriorityMgmt	OMX_PRIORITYMGMTTYPE

5.6 GetConfig()

Description

This function returns a configuration structure from a component.

You can call this function any time when the component is in the OMX_LoadedState state.

The nIndex parameter indicates the type of the structure being requested from the component.

- The application must allocate a corresponding structure and fill in the structure size and version information before calling this function.
- If the component has no parameter settings changed in the structure, the component returns the structure with default settings.

Some parameters may be invalid if they are not supported by the encoder hardware. For details, see [Chapter 2, Feature Support](#).

Syntax

```
OMX_ERRORTYPE GetConfig(  
    OMX_HANDLETYPE hComponent,  
    OMX_INDEXTYPE nIndex,  
    OMX_PTR pComponentConfigStructure  
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	nIndex	The index of the structure type of pComponentConfigStructure. For the list of supported structure types, see Section 5.6.1, Supported Structures .
IN/OUT	pComponentConfigStructure	A pointer to the structure that the component fills in.

Returns

OMX_ERRORTYPE

5.6.1 Supported Structures

The following tables list the structures supported for GetConfig() and their corresponding nIndex values in video and image domains, respectively.

Table 6. Structures Supported for GetConfig() in Video Domain

nIndex Value	Structure
OMX_IndexConfigCommonRotate	OMX_CONFIG_ROTATIONTYPE
OMX_IndexConfigCommonInputCrop	OMX_CONFIG_RECTTYPE
OMX_IndexConfigVideoFramerate	OMX_CONFIG_FRAMERATETYPE
OMX_IndexConfigVideoBitrate	OMX_VIDEO_PARAM_BITRATETYPE
OMX_IndexConfigCommonFrameStabilisation	(For H1 only) OMX_CONFIG_FRAMESTABTYPE
OMX_IndexConfigVideoIntraVOPRefresh	OMX_CONFIG_INTRAREFRESHVOPTYPE
OMX_IndexConfigVideoAVCIntraPeriod	OMX_VIDEO_CONFIG_AVCINTRAPERIOD
OMX_IndexConfigVideoVp8ReferenceFrame	(For H1 only) OMX_VIDEO_VP8REFERENCEFRAMETYPE
OMX_IndexConfigVideoAdaptiveRoi	(For H1 only) OMX_VIDEO_CONFIG_ADAPTIVEROITYPE
OMX_IndexConfigVideoVp8TemporalLayers	(For H1 only) OMX_VIDEO_CONFIG_VP8TEMPORALLAYERTYPE
OMX_IndexConfigVideoIntraArea	(For H1 only) OMX_VIDEO_CONFIG_INTRAAREATYPE
OMX_IndexConfigVideoRoiArea	(For H1 only) OMX_VIDEO_CONFIG_ROIAREATYPE
OMX_IndexConfigVideoRoiDeltaQp	(For H1 only) OMX_VIDEO_CONFIG_ROIDELTAQPTYPE
OMX_IndexConfigVideoIPCMArea	(For H1 only) OMX_VIDEO_CONFIG_IPCMAREATYPE

Table 7. Structures Supported for GetConfig() in Image Domain

nIndex Value	Structure
OMX_IndexConfigCommonRotate	OMX_CONFIG_ROTATIONTYPE
OMX_IndexConfigCommonInputCrop	OMX_CONFIG_RECTTYPE

5.7 SetConfig()

Description

This function sends a configuration structure to a component.

You can call this function once to send each configuration structure. You can call this function any time when the component is in the OMX_LoadedState state.

The nIndex parameter indicates the type of the structure being sent to the component.

- The application must allocate a corresponding structure and fill in the structure size and version information as well as other parameter settings before calling this function.
- The application is free to dispose of the structure after the function call, as the component has copied any data it retains.

Some parameters may be invalid if they are not supported by the encoder hardware. For details, see [Chapter 2, Feature Support](#).

Syntax

```
OMX_ERRORTYPE SetConfig(  
    OMX_HANDLETYPE hComponent,  
    OMX_INDEXTYPE nIndex,  
    OMX_PTR pComponentConfigStructure  
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	nIndex	The index of the structure type of pComponentConfigStructure. For the list of supported structure types, see Section 5.7.1, Supported Structures .
IN	pComponentConfigStructure	A pointer to the structure to be sent to the component.

Returns

OMX_ERRORTYPE

5.7.1 Supported Structures

The following tables list the structures supported for SetConfig() and their corresponding nIndex values in video and image domains, respectively.

Table 8. Structures Supported for SetConfig() in Video Domain

nIndex Value	Structure
OMX_IndexConfigCommonRotate	OMX_CONFIG_ROTATIONTYPE Note: This structure is configurable only through port 0.
OMX_IndexConfigCommonInputCrop	OMX_CONFIG_RECTTYPE Note: This structure is configurable only through port 0.
OMX_IndexConfigVideoFramerate	OMX_CONFIG_FRAMERATETYPE Note: This structure is configurable only through port 1.
OMX_IndexConfigVideoBitrate	OMX_VIDEO_PARAM_BITRATETYPE Note: This structure is configurable only through port 1.
OMX_IndexConfigCommonFrameStabilisation	(For H1 only) OMX_CONFIG_FRAMESTABTYPE Note: This structure is configurable only through port 0.
OMX_IndexConfigVideoIntraVOPRefresh	OMX_CONFIG_INTRAREFRESHVOPTYPE Note: This structure is configurable only through port 1.
OMX_IndexConfigVideoAVCIntraPeriod	OMX_VIDEO_CONFIG_AVCINTRAPERIOD Note: This structure is configurable only through port 1.
OMX_IndexConfigVideoVP8ReferenceFrame	(For H1 only) OMX_VIDEO_VP8REFERENCEFRAMETYPE Note: This structure is configurable only through port 1.
OMX_IndexConfigVideoAdaptiveRoi	(For H1 only) OMX_VIDEO_CONFIG_ADAPTIVEROITYPE
OMX_IndexConfigVideoVp8TemporalLayers	(For H1 only) OMX_VIDEO_CONFIG_VP8TEMPORALLAYERTYPE
OMX_IndexConfigVideoIntraArea	(For H1 only) OMX_VIDEO_CONFIG_INTRAAREATYPE
OMX_IndexConfigVideoIPCMArea	(For H1 only) OMX_VIDEO_CONFIG_IPCMAREATYPE
OMX_IndexConfigVideoRoiArea	(For H1 only) OMX_VIDEO_CONFIG_ROIAREATYPE
OMX_IndexConfigVideoRoiDeltaQp	(For H1 only) OMX_VIDEO_CONFIG_ROIDELTAQPTYPE

Table 9. Structures Supported for SetConfig() in Image Domain

nIndex Value	Structure
OMX_IndexConfigCommonRotate	OMX_CONFIG_ROTATIONTYPE Note: This structure is configurable only through port 0.
OMX_IndexConfigCommonInputCrop	OMX_CONFIG_RECTTYPE Note: This structure is configurable only through port 0.

5.8 GetExtensionIndex()

Description

This function translates a vendor-specific configuration or parameter string into an OMX structure index.

There is no requirement for the vendor to support this command for the indexes already found in the OMX_INDEXTYPE enumeration, which aims to save space in small components. The component shall support all vendor supplied extension indexes not found in the master OMX_INDEXTYPE enumeration.

Syntax

```
OMX_ERRORTYPE GetExtensionIndex(
    OMX_HANDLETYPE hComponent,
    OMX_STRING cParameterName,
    OMX_INDEXTYPE *pIndexType
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	cParameterName	The parameter name to be translated into a configuration index.
OUT	*pIndexType	A pointer to the structure used to receive the index value.

Returns

OMX_ERRORTYPE

5.8.1 Supported Extensions

The following table lists extensions supported by the encoders.

Note: Only H1 V6 encoders support the listed extension. You need to apply low-level implementation of Android native buffer allocation.

Table 10. Supported Extensions

cParameterName Value	Structure for Extension Configuration
OMX.google.android.index.prependSPSPSToIDRFrames	PrependSPSPSToIDRFramesParams

5.9 GetState()

Description

This function returns the current state of a component.

Syntax

```
OMX_ERRORTYPE GetState(
    OMX_HANDLETYPE hComponent,
    OMX_STATETYPE *pState
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
OUT	*pState	A pointer to a location used to receive the state.

Returns

OMX_ERRORTYPE

5.10 ComponentTunnelRequest()

Description:

This function checks whether tunneling between two components is possible and sets up the tunneling.

The return codes of this function are used to indicate whether tunneling is possible, or whether tunneling is supported.

When this function is called and nPort is an output port, the target component will populate the pTunnelSetup structure with the output port's requirements and constraints for the tunnel.

When this method is called and nPort is an input port, the target component will:

1. Query the necessary parameters from the output port to determine whether the ports are compatible for tunnelling.
2. Store the tunnel step provided by the output port if the ports are compatible.
3. Determine which port (either input or output) is the buffer supplier, and call OMX_SetParameter on the output port to indicate this selection.

Syntax

```
OMX_ERRORTYPE ComponentTunnelRequest(
    OMX_HANDLETYPE      hComp,
    OMX_U32              nPort,
    OMX_HANDLETYPE      hTunneledComp,
    OMX_U32              nTunneledPort,
    OMX_TUNNELSETUPTYPE *pTunnelSetup
);
```

Parameters

IN	hComp	The handle of the target component. This component also participates in the tunnel.
IN	nPort	The index of the hComp port used for tunneling.
IN	hTunneledComp	The handle of the other component that participates in the tunnel.
IN	nTunneledPort	The index of the hTunneledComp port used for tunneling.
IN/OUT	*pTunnelSetup	A pointer to the structure that contains data for tunneling negotiation between the components.

Returns

OMX_ERRORTYPE

5.11 UseBuffer()

Description

This function sends a request for the component to use a buffer already allocated by another component or by the IL Client, and allocates its own buffer header.

For information about the buffer payload formats and buffer sizes, see [Chapter 2, Feature Support](#).

Syntax

```
OMX_ERRORTYPE UseBuffer(
    OMX_HANDLETYPE          hComponent,
    OMX_BUFFERHEADERTYPE    **ppBuffer,
    OMX_U32                  nPortIndex,
    OMX_PTR                  pAppPrivate,
    OMX_U32                  nSizeBytes,
    OMX_U8                   *pBuffer
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN/OUT	**ppBuffer	A pointer to a pointer of a structure used to receive the pointer to the buffer header.
IN	nPortIndex	The index of the port to use the buffer, relative to the component that owns the port.
IN	pAppPrivate	The pointer to an implementation-specific memory location that is under responsibility of the supplier of the buffer.
IN	nSizeBytes	The buffer size in bytes.
IN	*pBuffer	A pointer to the buffer.

Returns

OMX_ERRORTYPE

5.12 AllocateBuffer()

Description

The function requests a component for allocating a new buffer and buffer header.

After this function is called, the component allocates the buffer and the buffer header and returns a pointer to the buffer header.

For information about the buffer payload formats and buffer sizes, see [Chapter 2, Feature Support](#).

Syntax

```
OMX_ERRORTYPE AllocateBuffer(
    OMX_HANDLETYPE      hComponent,
    OMX_BUFFERHEADERTYPE **ppBuffer,
    OMX_U32              nPortIndex,
    OMX_PTR              pAppPrivate,
    OMX_U32              nSizeBytes
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN/OUT	**ppBuffer	A pointer to a pointer of a structure used to receive the pointer to the buffer header.
IN	nPortIndex	The index of the port to use the buffer.
IN	pAppPrivate	The pointer to a structure used to initialize the pAppPrivate member of the buffer header structure.
IN	nSizeBytes	The buffer size in bytes.

Returns

OMX_ERRORTYPE

5.13 FreeBuffer()

Description

This function releases a buffer and buffer header from a component.

- If the buffer is allocated using **AllocateBuffer()**, then the component frees the buffer header.
- If the buffer is allocated using **UseBuffer()**, then the component frees the buffer and buffer header.

Syntax

```
OMX_ERRORTYPE FreeBuffer(
    OMX_HANDLETYPE      hComponent,
    OMX_U32             nPortIndex,
    OMX_BUFFERHEADERTYPE *pBufferHeader
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	nPortIndex	The index of the port using the buffer.
IN	*pBufferHeader	A pointer to a structure used to provide or receive the pointer to the buffer header.

Returns

OMX_ERRORTYPE

5.14 FillThisBuffer()

Description

The function sends an empty buffer to an output port of a component. Then, the component fills and returns the buffer to the application via the **FillBufferDone()** callback.

A call to this function is a non-blocking call in which the component records the buffer and returns the result immediately, and then fills the buffer later at the proper time.

This function can be called only if the component is in the **OMX_ExecutingState** state.

If **nPortIndex** does not specify an output port, the component returns an error.

Note: If usage of externally allocated frame buffers is enabled (using `-DUSE_EXTERNAL_BUFFER -DUSE_OUTPUT_RELEASE`), the client must send a `nBufferCountMin` number of buffers to the component using `FillThisBuffer()` call and wait for `FillBufferDone()` callback before the additional output buffers (`nBufferCountActual`) are sent to the component.

Syntax

```
OMX_ERRORTYPE FillThisBuffer(
    OMX_HANDLETYPE      hComponent,
    OMX_BUFFERHEADERTYPE *pBuffer
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	*pBuffer	A pointer to the structure used to provide or receive the pointer to the buffer header.

Returns

OMX_ERRORTYPE

5.15 EmptyThisBuffer()

Description

This function sends a filled buffer to an input port of a component. Then, the component empties and returns the buffer to the application via the `EmptyBufferDone()` callback.

A call to this function is a non-blocking call in which the component records the buffer and returns the result immediately, and then empties the buffer later at the proper time.

This function can be called only if the component is in the `OMX_ExecutingState` state.

If `nPortIndex` does not specify an input port, the component returns an error.

Syntax

```
OMX_ERRORTYPE EmptyThisBuffer(
    OMX_HANDLETYPE      hComponent,
    OMX_BUFFERHEADERTYPE *pBufferHeader
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	*pBufferHeader	A pointer to the structure used to provide or receive the pointer to the buffer header.

Returns

OMX_ERRORTYPE

5.16 SetCallbacks()

Description

This function allows the core to transfer the callback structure from the application to the component.

Syntax

```
OMX_ERRORTYPE SetCallbacks (
    OMX_HANDLETYPE hComponent,
    OMX_CALLBACKTYPE *pCallbacks,
    OMX_PTR pAppData
);
```

Parameters

IN	hComponent	The handle of the component to execute the function.
IN	*pCallbacks	A pointer to the structure used to provide the callback information to the component.
IN	pAppData	A pointer to a value that the application defines to allow the application to determine the context of the call.

Returns

OMX_ERRORTYPE

5.17 ComponentDeInit()

Description

This function de-initializes a component to free the resources allocated at component initialization.

After this function is called, the component handle is no longer valid.

Syntax

```
OMX_ERRORTYPE ComponentDeInit (  
    OMX_HANDLETYPE  hComponent  
);
```

Parameters

IN	hComponent	The handle of the component to be de-initialized.
----	------------	---

Returns

OMX_ERRORTYPE

Document Revision History

This section describes top level differences in the versions of this document.

Note: This document is not necessarily updated for each patch or minor revision. The information in this document tends to be stable across a revision (*nnn*) series.

Document Revision	Date	Compatible Core	Description
0.81	2023-12-31	- Hantro H1 Video Encoder V7 or later - Hantro H2 Video Encoder V3 or later - Hantro VC8000E Series Video Encoder - Hantro VC9000E Series Video Encoder	- Updated the document name. - Made miscellaneous refinements.
0.80	2023-12-05	- Hantro H1 Video Encoder V7 or later - Hantro H2 Video Encoder V3 or later - Hantro VC8000E Series Video Encoder - Hantro VC9000E Series Video Encoder	Initial release.